

STRING E ALOCAÇÃO DE MEMÓRIA

Prof. André Backes

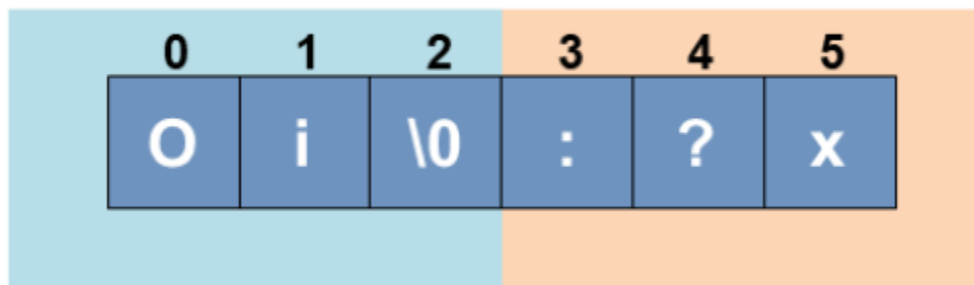
STRING

Strings

- São sequências de caracteres utilizadas para representar palavras, frases e textos completos
 - C++ suporta strings no padrão C: array de caracteres
 - Seu funcionamento é idêntico ao da linguagem C

```
char str[6] = "Oi";
```

Região inicializada: 2
letras + 1 caractere
terminador '\0'



Lixo de memória
(região não
inicializada)

Classe std::string

- É o padrão para manipular strings no C++
 - Manipula sequências de caracteres de forma fácil e rápida
 - Definida na biblioteca biblioteca <string>
 - Uma string é um objeto e não apenas um array de caracteres
- Forma geral:

```
string str;
```

Inicialização e atribuição da string

- A inicialização pode ser feita usando
 - O operador de atribuição =

```
string str = "texto";
```

- O construtor da classe string

```
string str("texto");
```

- Podemos usar o operador = para atribuir o conteúdo de uma string a outra

```
string str1 = "linguagem C++";  
string str2, str3;
```

```
str2 = "texto";  
str3 = str1;
```

Acessando os elementos da string

- Mesmo sendo um objeto, podemos acessar cada caractere contido na string por meio do operador de colchetes

```
string str = "Linguagem C++";  
  
for(int i=0; i< str.size(); i++)  
    cout << str[i] << endl;
```

Escrevendo/lendo uma string

- Para fazer a saída/escrita de uma `std::string` utilizamos o objeto **cout**

```
string str = "Hello World!";  
  
cout << "string lida: " << str << endl;
```

- Podemos ler uma string usando o objeto `cin`
 - Infelizmente o objeto **cin** lê apenas strings digitadas sem espaços, ou seja, palavras

```
string str;  
cin >> str;  
cout << "string lida: " << str << endl;
```

Lendo uma string do teclado

- Um modo mais adequado para ler uma string é usando o método **getline()**

```
string str;  
getline(cin, str);  
cout << "string lida: " << str << endl;
```

- Neste caso, a leitura termina quando um caractere de nova linha ('\n' ou ENTER) é encontrado

Limpando o buffer do teclado

- As vezes, podem ocorrer erros durante a leitura de caracteres ou strings do teclado
- Para resolver isso, podemos limpar o buffer do teclado usando o método **ignore()** do objeto **cin**

```
string str;  
getline(cin, str);  
/// descarta o restante do buffer  
cin.ignore(32767, '\n');
```

Manipulando strings

- O objeto string possui alguns métodos internos especialmente desenvolvidos para a manipulação de caracteres

Tamanho de uma string

- Para se obter o tamanho de uma string, podemos usar os métodos **length()** ou **size()**
 - Ambos retornam o número de caracteres que existem antes do caractere '\0', e não o tamanho do array interno onde a string está armazenada

```
string str = "Hello World!";  
  
cout << str.length() << endl;  
  
cout << str.size() << endl;
```

Copiando uma string

- Podemos fazer a cópia de strings usando o operador de atribuição “=” ou o método interno **assign()**:

```
string str1, str2 = "Hello World!";  
  
str1.assign(str2);  
  
str1 = str2;
```

Concatenando strings

- Consistem em copiar uma string (ou caractere) para o final de outra string
- Para essa tarefa podemos usar
 - O operador de soma “+”
 - O método interno **append(string)**
 - Ou o método interno **push_back(caractere)**

```
string str1 = "Bom";  
string str2 = "dia";
```

```
str1 += " ";  
str1.append(str2);  
str1.push_back('!');
```

Comparando duas strings

- A linguagem C++ permite o uso dos operadores de relacionais para comparar duas strings
 - >, <, >=, <=, == e !=
- O resultado dessa comparação é um valor booleano (true ou false)

```
string str1 = "Texto A";  
string str2 = "Texto B";  
  
if(str1 == str2)  
    cout << "Iguais" << endl;  
else  
    cout << "Diferentes" << endl;
```

Comparando duas strings

- Também é possível usar o método **compare()** para comparar duas strings
 - Esse método retorna um valor inteiro igual a zero no caso das duas strings serem iguais

```
string str1 = "Texto A";  
string str2 = "Texto B";  
  
if(str1.compare(str2) ==0)  
    cout << "Iguais" << endl;  
else  
    cout << "Diferentes" << endl;
```

PONTEIROS E ALOCAÇÃO DE MEMÓRIA

C vs C++ | ponteiros

- Ponteiros funcionam da mesma forma em ambas as linguagens
 - Operações aritméticas, relacionais, ponteiros genéricos, para funções etc.

Linguagem C

```
int idade = 21;
int *p;

p = &idade;

printf("%d\n", *p);
printf("%p\n", p);
```

Linguagem C++

```
int idade = 21;
int *p;

p = &idade;

cout << *p << endl;
cout << p << endl;
```

C vs C++ | ponteiros

- A linguagem C++ não possui a macro **NULL** para definir um ponteiro nulo
 - Isso é feito atribuindo o valor **ZERO** ao ponteiro
 - O padrão C++11 possui a palavra chave **nullptr** para definir o valor de um ponteiro como nulo

```
///Usando 0 (ZERO)
```

```
int *p1 = 0;
```

```
///Usando nullptr (C++11)
```

```
float *p2;
```

```
p2 = nullptr;
```

```
///Usando NULL (padrão C, não é do C++)
```

```
double *p3 = NULL;
```

Operador de referência &

- A linguagem C++ possui um outro tipo especial de variável chamado de **referência**
 - Basicamente, é uma variável que funciona como um sinônimo para outra variável já existente
- Forma geral

```
tipo_referência &nome_referência = variável;
```

Operador de referência &

- Referências devem ser sempre inicializadas
 - Podemos pensar nelas como um ponteiro constante
 - Devemos informar para a referência a variável da qual ela será um sinônimo na sua criação

```
int idade = 22;
int &refe = idade;

cout << "valor = " << idade << endl;
cout << "valor = " << refe << endl;

refe = 27;
cout << "valor = " << idade << endl;
cout << "valor = " << refe << endl;

cout << "endereco = " << &idade << endl;
cout << "endereco = " << &refe << endl;
```

Saída:

```
valor = 22
valor = 22
valor = 27
valor = 27
endereco = 0x69fee8
endereco = 0x69fee8
```

Operador de referência &

- Não podemos mudar a referência de lugar
 - A referência sempre será um sinônimo para a variável definida na sua inicialização

```
int idade = 22;
int total = 100;
int &refe = idade;

cout << "valor = " << idade << endl;
cout << "valor = " << refe << endl;

refe = total;
cout << "valor = " << idade << endl;
cout << "valor = " << refe << endl;
```

Saída:

```
valor = 22
valor = 22
valor = 100
valor = 100
```

Operador de referência &

- Referências podem ser usadas com estruturas
 - Também podemos criar uma referência para um dos campos da struct (como um atalho)

```
struct ponto{  
    int x, y;  
};  
  
struct quadrado{  
    ponto p1, p2;  
};
```

```
int main(){  
    ponto p = {4,5};  
    ponto &refP = p;  
    cout << "valor = " << p.x << endl;  
    cout << "valor = " << refP.x << endl;  
  
    quadrado q;  
    int &refX = q.p1.x;  
  
    refX = 10;  
    cout << "valor = " << q.p1.x << endl;  
    cout << "valor = " << refX << endl;  
  
    return 0;  
}
```

Operador de referência &

- Podemos usar **const** com uma referência
 - A referência declarada aponta para um valor constante
 - No padrão C++11, uma referência constante pode armazenar um valor ou resultado de uma expressão

```
int total = 100;  
const int &ref1 = 100;  
cout << "valor = " << ref1 << endl;  
ref1 = 10; /// Erro
```

```
const int &ref2 = 2 + 3;  
cout << "valor = " << ref2 << endl;  
ref2 = 10; /// Erro
```

Alocação Dinâmica

- Variável
 - É uma posição de memória que armazena um dado que pode ser usado pelo programa
 - Deve ser declarada antes de ser usada

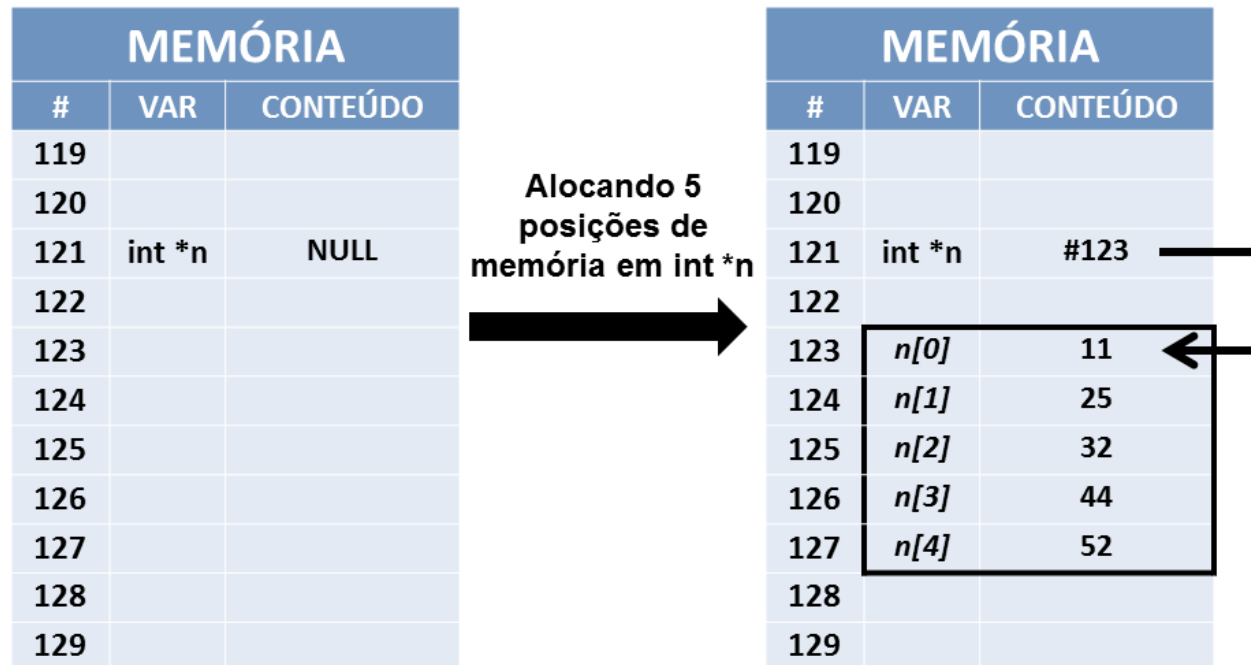
```
int main() {  
    int x;  
    float y;  
  
    return 0;  
}
```


Alocação Dinâmica

- Nem sempre é possível saber o quanto de memória um programa irá precisar
 - Qual o tamanho da imagem que o Photoshop irá manipular?
 - Podemos declarar uma matriz absurdamente grande prevendo um tamanho máximo?

Alocação Dinâmica

- A linguagem C++ permite alocar (reservar) dinamicamente (em tempo de execução) blocos de memórias utilizando ponteiros
 - A isso dá-se o nome de alocação dinâmica



Alocação Dinâmica | C vs C++

Linguagem C

- 4 funções disponíveis
 - malloc()
 - calloc()
 - realloc()
 - free()

Linguagem C++

- 4 operadores disponíveis
- Alocação de variável:
 - new e delete
- Alocação de array unidimensional
 - new[] e delete[]

Alocando e liberando memória

- Em C++, para alocar memória usamos o operador **new**

`ponteiro = new tipo_dado;`

- Para liberar a memória utilizamos o operador **delete**

`delete ponteiro;`

```
int main(){  
    // Faz a alocação de memória  
    int *p = new int;  
    // Atribui um valor ao conteúdo  
    *p = 78;  
  
    cout << "Endereco = " << p << endl;  
    cout << "Conteudo = " << *p << endl;  
  
    // Libera a memória alocada  
    delete p;  
  
    return 0;  
}
```

Alocando e liberando memória

- O operador **new** pode ser utilizado para alocar qualquer tipo de variável

```
struct aluno{
    string nome;
    int matricula;
    int notas[3];
};

int main(){
    // Aloca memória para um int
    int *p1 = new int;
    // Aloca memória para um double
    double *p2 = new double;
    // Aloca memória para uma struct aluno
    aluno *p3 = new aluno;
    // Aloca memória para um int*
    int **p4 = new int*;

    delete p1;
    delete p2;
    delete p3;
    delete p4;

    return 0;
}
```

Alocando e liberando memória

- Cuidado, a alocação de memória pode falhar
 - Podemos adicionar o comando **(std::nothrow)** para que o mesmo retorne NULL no caso de erro

```
int main() {  
    int *p = new (std::nothrow) int;  
    if(!p) {  
        cout << "Erro: Memoria Insuficiente!";  
    } else {  
        *p = 78;  
        cout << "Conteudo = " << *p << endl;  
        delete p;  
    }  
  
    return 0;  
}
```

Alocando e liberando memória

- Apenas libere a memória quando tiver certeza de que ela não será mais usada
 - Do contrário, erros podem acontecer ou o programa poderá não funcionar como esperado

```
int main() {  
    int *p = new int (100);  
  
    cout << *p << endl;  
  
    delete p;  
  
    ///Comportamento inesperado!  
    cout << *p << endl;  
  
    return 0;  
}
```

Iniciando a memória alocada

- A linguagem C++ possui três formas diferentes de inicializar a memória alocada
 - Inicialização com zero

```
int *p1 = new int ();
```

- Inicialização direta, com parênteses ()

```
int *p2 = new int (78);
```

- Inicialização uniforme (padrão C++11), com chaves {}:

```
int *p3 = new int {100};
```


Alocando e liberando arrays

- Diferente da linguagem C, C++ trata diferente a alocação de variáveis e arrays

- Para alocar memória para um array unidimensional usamos o operador **new []**

ponteiro = new tipo_dados[tamanho];

- Para liberar um array previamente alocado utilizamos o operador **delete[]**

delete[] ponteiro;

```
int main(){
    int *vet, N, soma = 0;
    cout << "Digite N: ";
    cin >> N;
    /// Faz a alocação de memória
    vet = new int[N];

    for(int i=0; i<N; i++){
        cout << "Digite um valor: ";
        cin >> vet[i];
    }
    /// Libera a memória alocada
    delete[] vet;

    return 0;
}
```

Inicializando o array alocado

- A linguagem C++ possui duas formas diferentes de inicializar a memória alocada
 - Inicialização com zero

```
int *v1 = new int[3] ();
```

- Inicialização uniforme (padrão C++11), com chaves {}:

```
int *v2 = new int[3] {10, 20, 30};
```

Alocação de arrays multidimensionais

- Existem várias soluções na linguagem C++ para se alocar um array com mais de uma dimensão
- Algumas delas
 - Usando o entendimento padrão de array
 - Usando ponteiro para ponteiro

Alocação de arrays multidimensionais

- Usando o entendimento padrão de array
 - O operador **new[]** permite definir apenas a primeira dimensão do array em tempo de execução
 - As demais devem ser constantes

```
int main() {  
  
    int N;  
    cout << "Digite o valor de N: ";  
    cin >> N;  
  
    auto matriz = new int[N][4];  
    auto cubo = new int[N][5][3];  
  
    delete[] matriz;  
    delete[] cubo;  
  
    return 0;  
}
```

Alocação de arrays multidimensionais

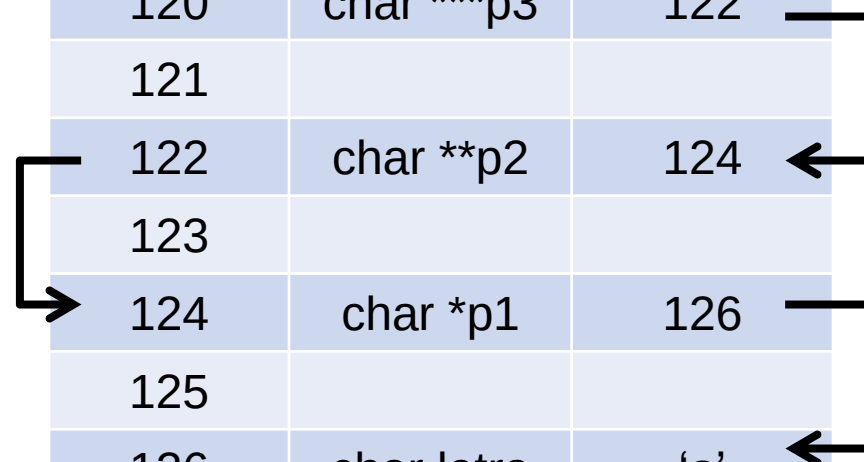
- Usando ponteiro para ponteiro
 - Em um ponteiro para ponteiro, cada nível do ponteiro permite criar uma nova dimensão no array
 - **int***
 - permite criar um array de **int**
 - **int****
 - permite criar um array de **int***
 - Cada posição pode ser novamente alocada, criando um array de **int**

Alocação de arrays multidimensionais

- Usando ponteiro para ponteiro

```
char letra = 'a';  
char *p1;  
char **p2;  
char ***p3;  
  
p1 = &letra;  
p2 = &p1;  
p3 = &p2;
```

Memória		
posição	variável	conteúdo
119		
120	char ***p3	122
121		
122	char **p2	124
123		
124	char *p1	126
125		
126	char letra	'a'
127		



```
graph TD
    p3[120: char ***p3] --> p2[122: char **p2]
    p2 --> p1[124: char *p1]
    p1 --> letra[126: char letra]
```

Alocação de arrays multidimensionais

- Usando ponteiro para ponteiro

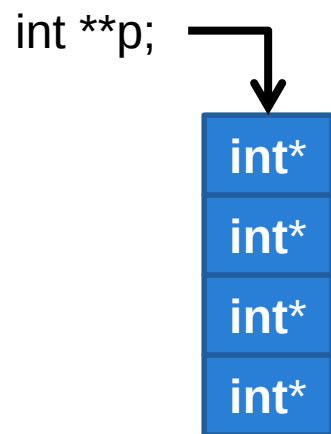
```
int main() {  
  
    int **p; ///  
    int N = 2;  
    ///  
    p = new int*[N];  
    for(int i = 0; i < N; i++){  
        p[i] = new int[N];  
        for(int j = 0; j < N; j++){  
            cin >> p[i][j];  
        }  
  
    ///  
    for(int i = 0; i < N; i++){  
        delete[] p[i];  
    }  
    delete[] p;  
  
    return 0;  
}
```

Memória		
posição	variável	conteúdo
119	int **p	120
120	p[0]	123
121	p[1]	126
122		
123	p[0][0]	69
124	p[0][1]	74
125		
126	p[1][0]	14
127	p[1][1]	31
128		

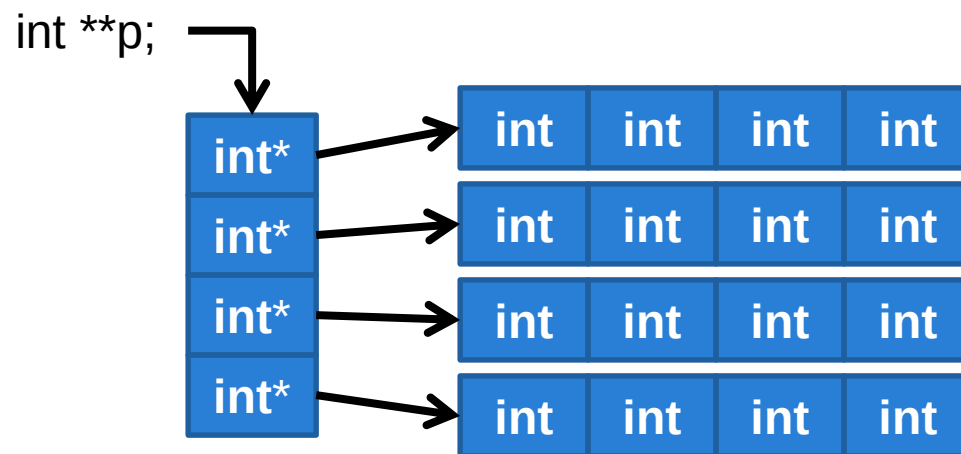
Alocação de arrays

- Em um ponteiro para ponteiro, cada nível do ponteiro permite criar uma nova dimensão no array.

1º new:
cria as linhas



2º new:
cria as colunas



Material Complementar

- Aula 07 - String: conceitos básicos
 - <https://youtu.be/sBWUSy4vWMY>
- Aula 08 - String: leitura e escrita
 - <https://youtu.be/Q12id6Mgocg>
- Aula 09 - String: cópia, concatenação e comparação
 - <https://youtu.be/ZfJeDo-mXeA>
- Aula 10 - Ponteiro e referência
 - <https://youtu.be/uiE67yeS0pA>
- Aula 14 - Alocação dinâmica de variável
 - <https://youtu.be/FgTKfllL1dg>
- Aula 15 - Alocação dinâmica de array
 - <https://youtu.be/4Qs7TdwfdFY>
- Aula 16 - Alocação dinâmica de matrizes
 - <https://youtu.be/gxKuppBnbNE>